Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense(128, ReLU)} \rightarrow \text{Dense(64, ReLU)} \rightarrow \text{Dense(10, Softmax)}$$

4. Dataset

Dataset: Fashion-MNIST Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images. **Task 2: Data Preprocessing**

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`. **Task 4: Model Training** Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32. **Task 5: Model Evaluation** Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Source Code

https://github.com/ashrithaxx/Deep-Learning-Multi-Layer-Perceptron

8. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper. Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

9. Mandatory Plots

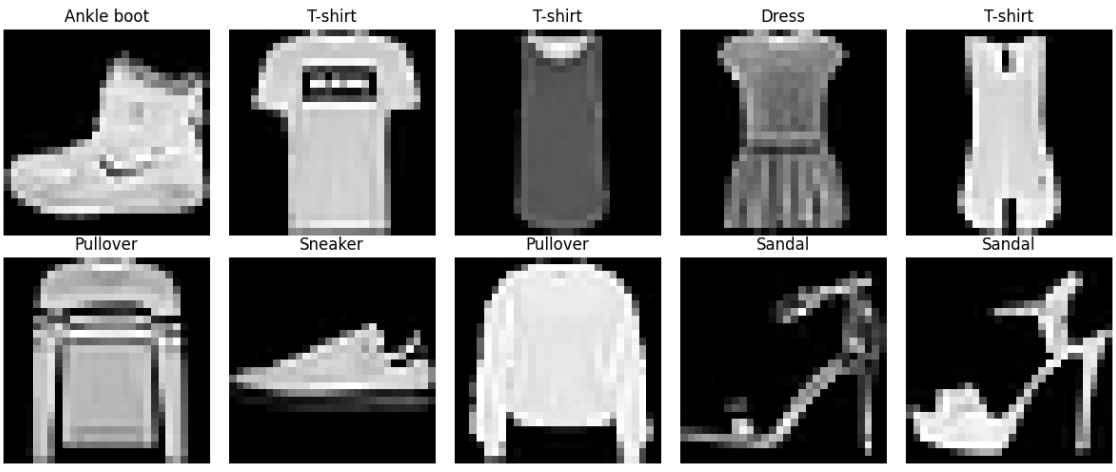

Figure 1: Sample images from the Fashion-MNIST dataset

This contains sample images from the dataset with items like sandal, dress, pullover etc. The images seem to have low resolution.

Figure 2: Class distribution across the training set

All the classes contain 6000 training images making it a perfectly balanced dataset. So accuracy is a fair evaluation metric for this experiment.

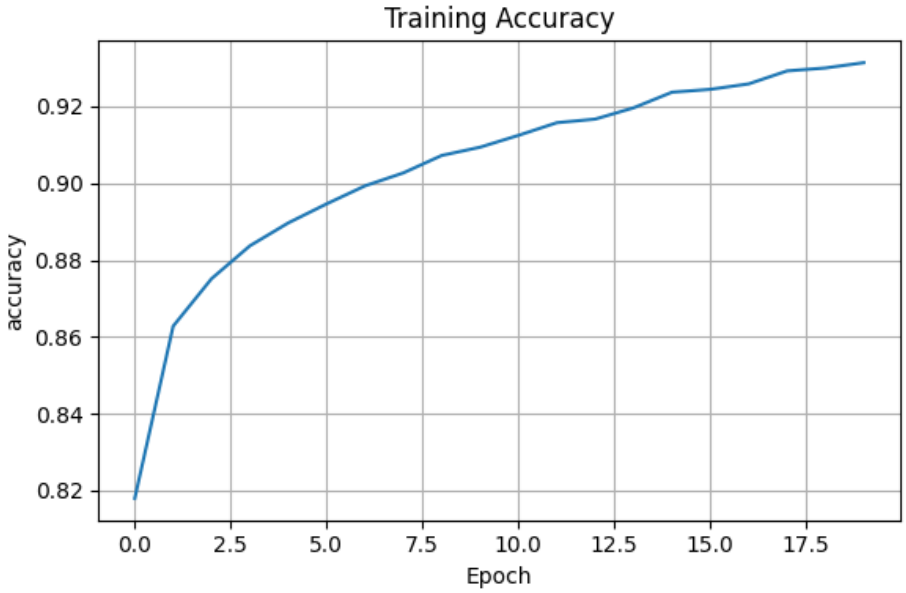

Figure 3: Training accuracy across epochs

Training accuracy rises from about 0.82 to 0.93 over 20 epochs without any sudden jumps, indicating that the Adam optimizer was converging smoothly and the learning rate was correct for this model.

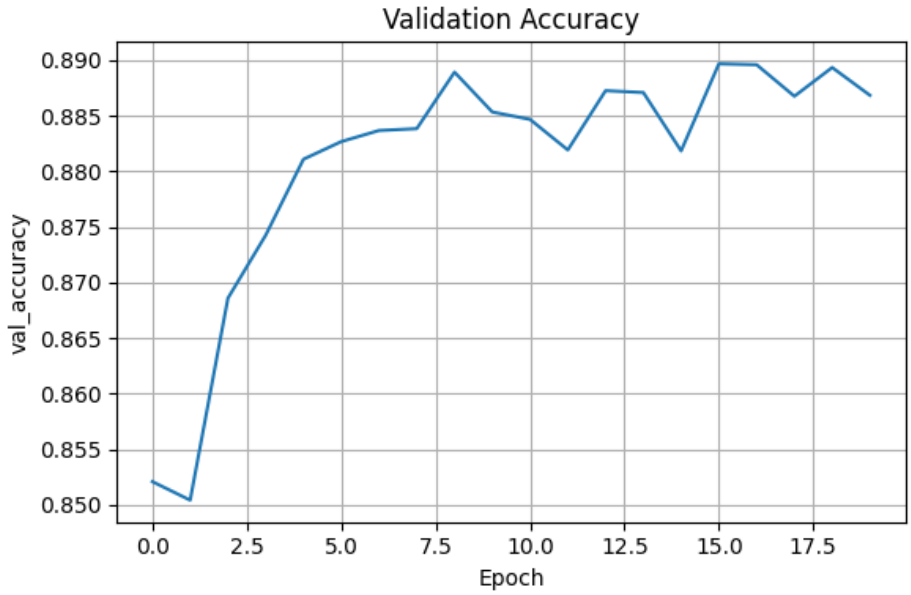

Figure 4: Validation accuracy across epochs

Validation accuracy grows quickly in the first five epochs and then is unstable around 0.885-0.89 for the rest of training.

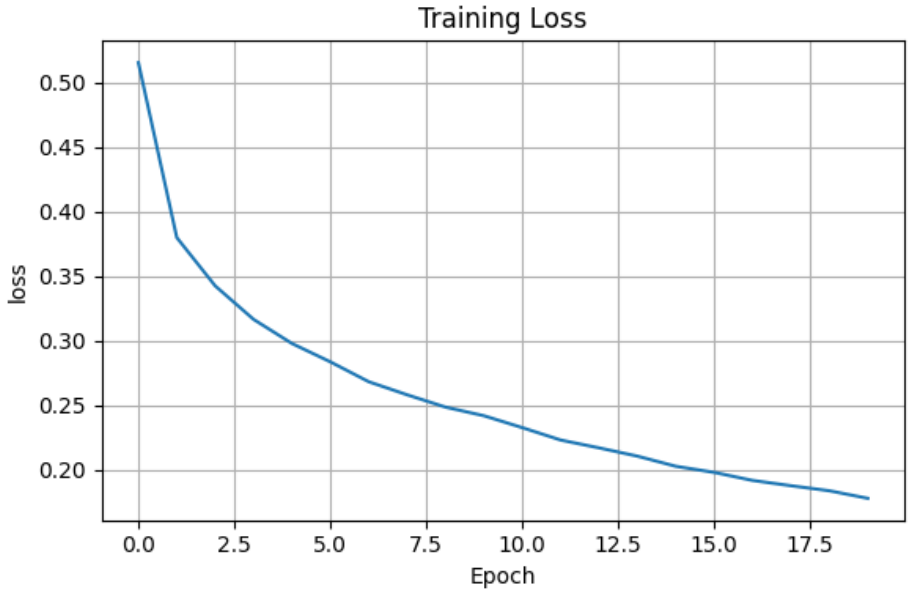

Figure 5: Training loss across epochs

Training loss decreases consistently throughout the 20 epochs, which is expected as the model learns..

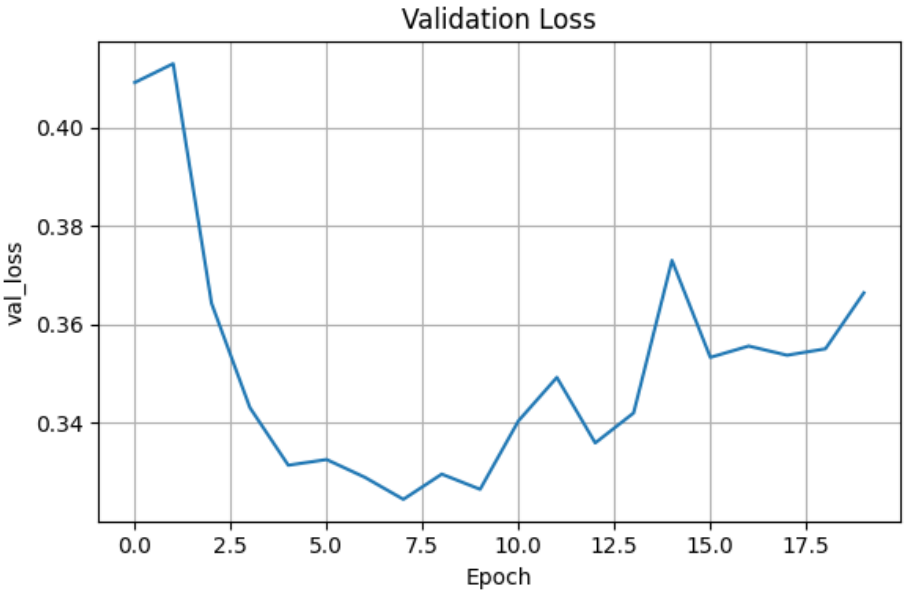

Figure 6: Validation loss across epochs

Unlike the training loss, validation loss bottoms out around epoch 9 and then goes upward slightly for the remaining epochs. The increase in validation loss after epoch 9 indicates slight overfitting.

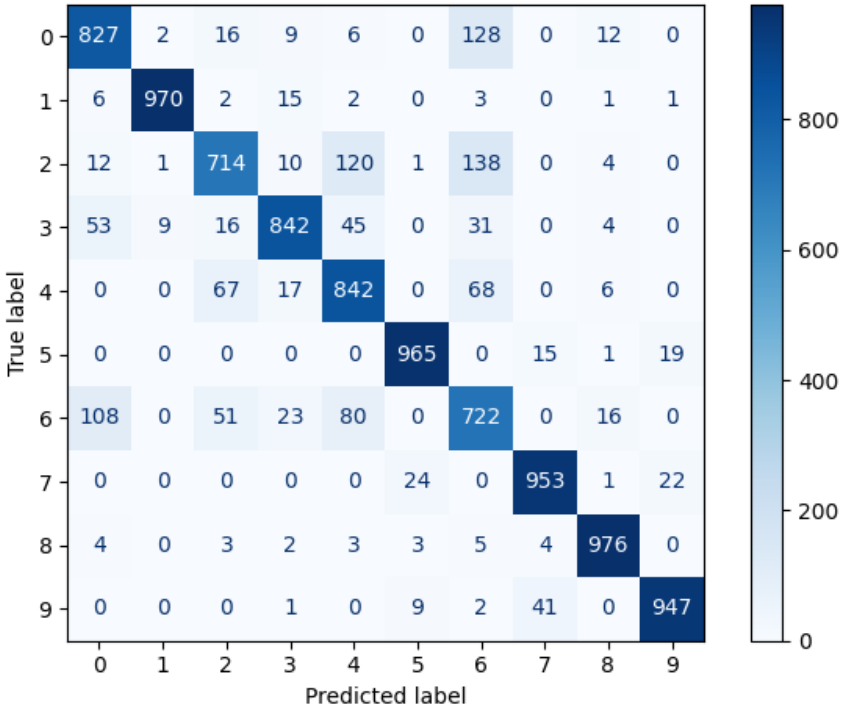

Figure 7: Confusion matrix – baseline model

The baseline model separates most classes well, but there was confusion between Shirt and Tshirt/Pullover/Coat due to the similarity of the images. Trouser, Sandal, Bag and Ankle boot are classified almost perfectly.

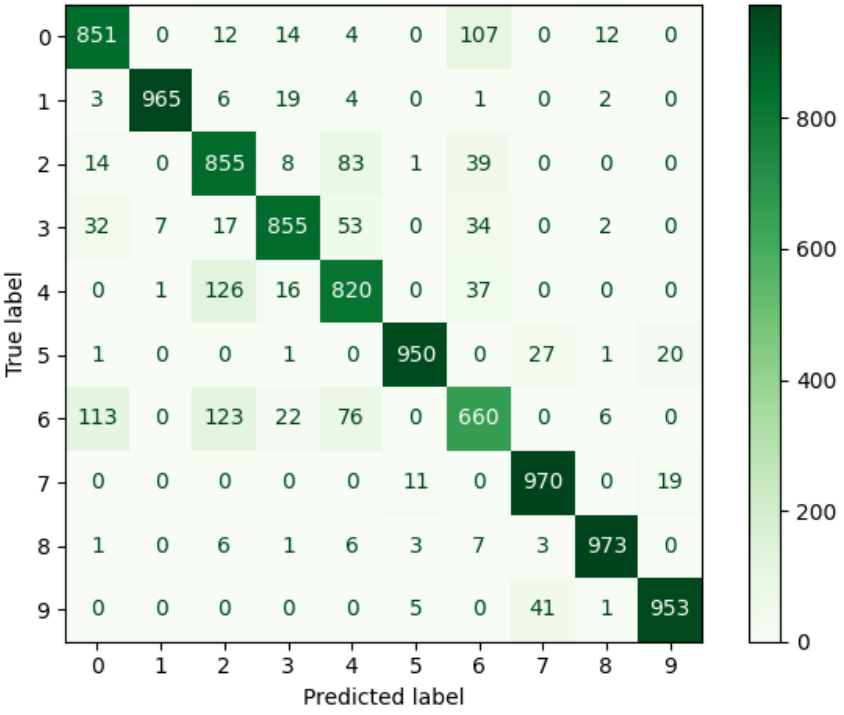

Figure 8: Confusion matrix – optimized model

The optimized model correctly classifies more Tshirts than the baseline but Shirt is still the weakest class. Overall the confusion pattern is very similar to the baseline.

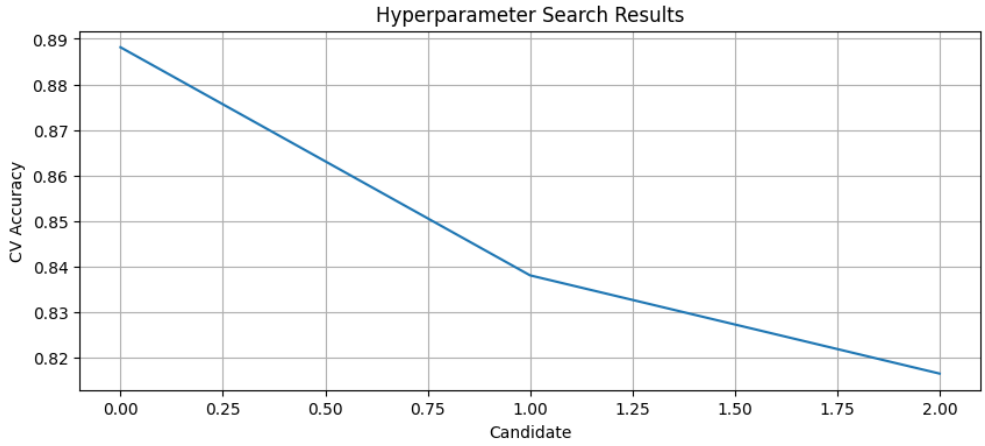

Figure 9: Cross-validation accuracy across hyperparameter search candidates

The Cross-validation accuracy drops sharply after the top candidate, dropping from about 0.888 to 0.816. This decline suggests that there were poor combinations (most likely high learning rates or too few hidden neurons).

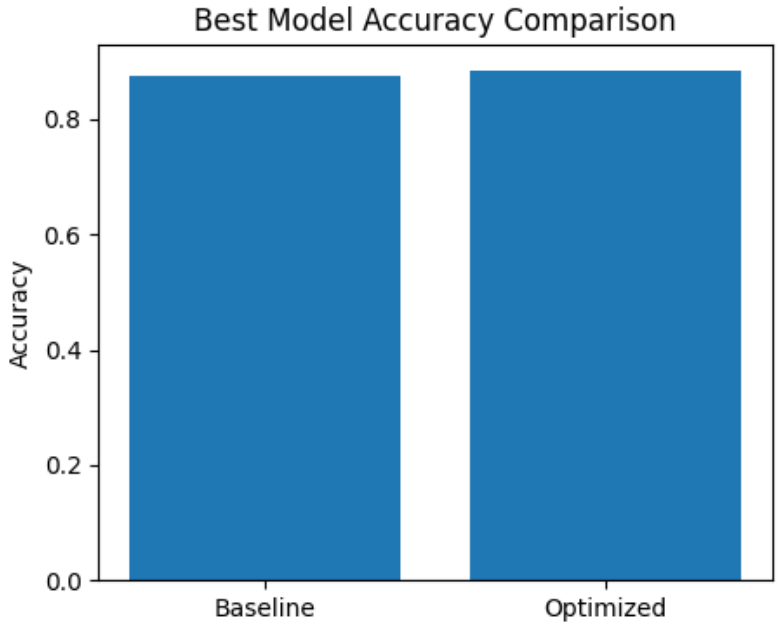

Figure 10: Baseline vs optimized model accuracy

The bar chart shows the optimized model outperforming the baseline in test accuracy (0.884 vs 0.876).

10. Results

Best Hyperparameters

Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	RMSProp
Activation Function	Tanh
Epochs	30
Dropout	0.2
Cross-validation Accuracy	0.8882
Testing Accuracy	0.8840

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8758	0.8840
Precision	0.8787	0.8849
Recall	0.8758	0.8840
F1-score	0.8764	0.8840
Training Time	144.79 s	212.13 s

11. Discussion

Answer the following:

1. **Which optimization method was used?** RandomizedSearchCV with 5-fold cross-validation using the SciKeras wrapper.
2. **Which hyperparameters were selected?** Hidden Layers: 3, Hidden Neurons: 128, Learning Rate: 0.001, Batch Size: 32, Optimizer: RMSProp, Activation Function: Tanh, Epochs: 30, Dropout Rate: 0.2
3. **Did optimization improve performance?** Yes, but only slightly. Test accuracy improved from 0.8758 to 0.8840. The trade-off is training time, which nearly doubled (144.79s to 212.13s) due to the extra hidden layer and higher epoch count.
4. **Which hyperparameter had the greatest impact?** The learning rate had the strongest effect on model performance. The optimal value of 0.001 produced the highest cross-validation accuracy and more stable training.
5. **Compare Grid Search and Randomized Search.** Grid Search tests all combinations while Randomized Search tests only a subset, reducing computation time while maintaining good performance.
6. **Recommend the best MLP configuration.** The best model uses 3 hidden layers, 128 neurons, Tanh activation, RMSProp, learning rate 0.001, batch size 32, dropout 0.2 and 30 epochs providing the best overall performance.

12. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

13. Conclusion

The optimized TensorFlow/Keras MLP improved test accuracy from 87.58% to 88.40%, at the cost of longer training time.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.

Additional Task

XOR Gate

The perceptron was trained on the XOR dataset. Table 1 shows the weights and bias after each update. Since the XOR problem is not linearly separable, the perceptron repeatedly updates its weights without converging.

Table 1: Weight Updates for XOR Gate

Update	Weights (w_1, w_2)	Bias
1	(0,0)	-1
2	(0,1)	0
3	(-1,0)	-1
4	(-1,1)	0
5	(0,1)	1
6	(-1,0)	0
7	(-1,0)	-1
8	(-1,1)	0
9	(0,1)	1
10	(-1,0)	0
11	(-1,0)	-1
12	(-1,1)	0
13	(0,1)	1
14	(-1,0)	0
15	(-1,0)	-1
16	(-1,1)	0
17	(0,1)	1
18	(-1,0)	0
19	(-1,0)	-1
20	(-1,1)	0
21	(0,1)	1
22	(-1,0)	0
23	(-1,0)	-1
24	(-1,1)	0
25	(0,1)	1
26	(-1,0)	0
27	(-1,0)	-1
28	(-1,1)	0
29	(0,1)	1
30	(-1,0)	0
31	(-1,0)	-1
32	(-1,1)	0
33	(0,1)	1
34	(-1,0)	0
35	(-1,0)	-1
36	(-1,1)	0
37	(0,1)	1
38	(-1,0)	0

The perceptron did not converge after 10 epochs because the XOR dataset is not linearly separable. Instead, the weights repeatedly cycle through the same values.

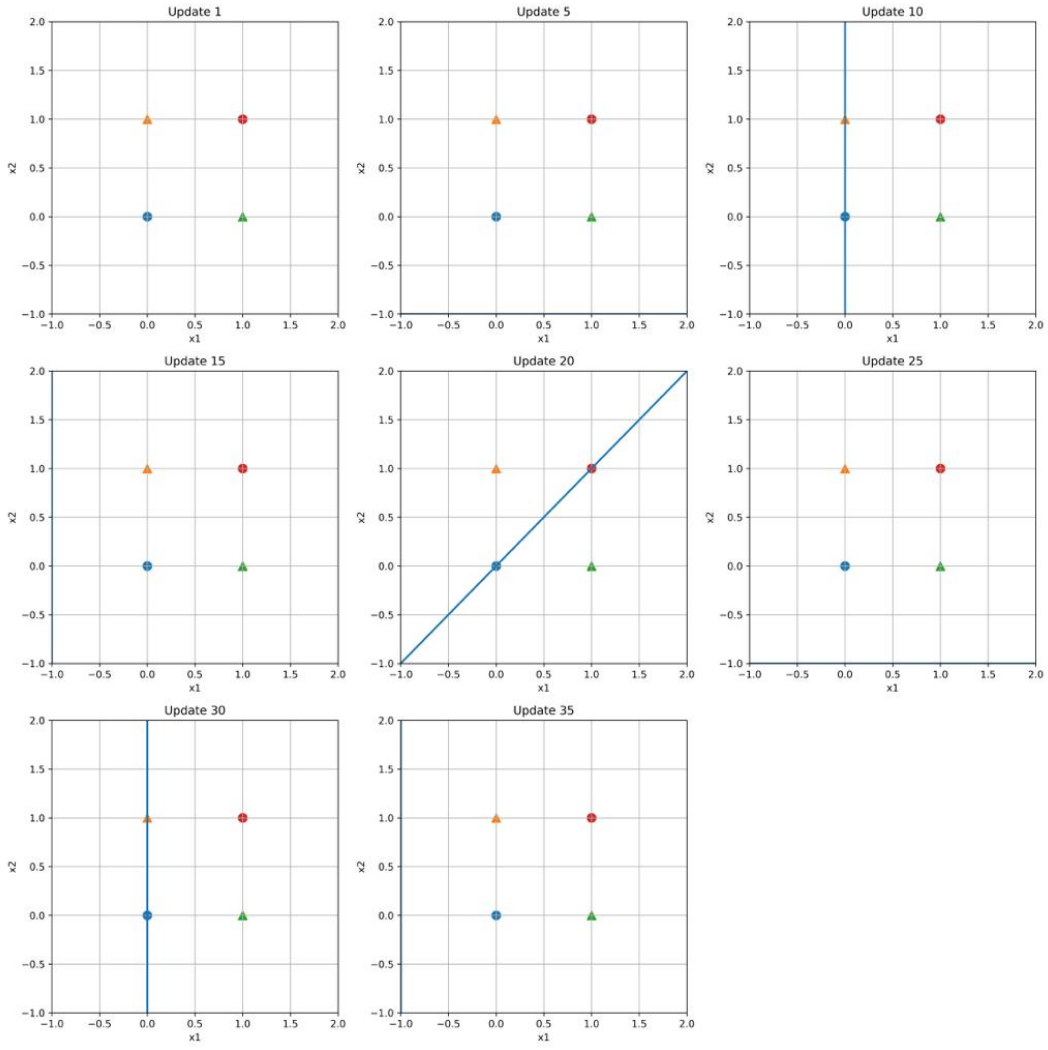

Figure 1: Decision Boundary Evolution for XOR Gate (Update 1 and Every 5th Update)